Room Discovery App — Complete API Design (All Phases)

Base path: `/api/v1`. Auth = "required" means a valid session/JWT from your custom OTP flow (see Phase 1 → Auth). "optional" means the route works unauthenticated but behaves differently if a user is present.

Build order reminder: **design here does not mean build now.** Phase 2 and 3 sections exist so you can see the whole shape of the system and avoid painting yourself into a corner in Phase 1 — not as a signal to start coding them early.

PHASE 1 — MVP (build this first, launch on this alone)

Auth

Method	Path	Auth	Rate limit	Notes
POST	`/auth/otp/request`	none	5 / phone / hour, also keyed by IP	Sends OTP via third-party SMS (Azure can't reach Indian numbers). Same response whether or not the number is already registered — no enumeration.
POST	`/auth/otp/verify`	none	5 attempts / phone / 15 min, then 1hr lockout	Verifies code, creates/links `users` row, issues session token.
POST	`/auth/email/login`	none	5 / email / hour	Alternative to phone.
GET	`/auth/me`	required	—	Current user's profile.
POST	`/auth/logout`	required	—	Invalidate session.

Users

Method	Path	Auth	Notes
GET	`/users/me`	required	Full own profile.

Method	Path	Auth	Notes
PATCH	`/users/me`	required	Update name/photo. Phone or email change re-triggers OTP, not a plain field edit.
GET	`/users/:id`	optional	Public-safe subset only — never expose another user's phone/email directly.

Rooms

Method	Path	Auth	Rate limit	Notes
POST	`/rooms`	required	5 new listings / user / day	Creates with `status = pending_review`.
GET	`/rooms/:id`	optional	—	Full detail incl. images array. Non-active room → 404 for non-owners, not 403.
PATCH	`/rooms/:id`	required, owner	—	Resets `status` to `pending_review` on any content edit.
DELETE	`/rooms/:id`	required, owner	—	Soft delete → `status = inactive`. Never hard-delete (keeps `interests` history).
GET	`/rooms`	optional	—	Filters: `city`, `room_type`, `min_rent`, `max_rent`, `page`, `limit`. Forces `status = active` for non-owners.
GET	`/rooms/nearby`	optional	—	Params: `lat`, `lng`, `radius_km` (default 5, cap 25), `limit`. Core value prop — the PostGIS `<->` + `ST_DWithin` query.
POST	`/rooms/:id/view`	optional	1 / session / room / hour	Increments `view_count`, deduped so refresh-spam doesn't inflate it.

Room Images

Method	Path	Auth	Notes
POST	`/rooms/:id/images`	required, owner	Returns presigned upload URL (Blob Storage/S3). Client uploads directly — never proxy image bytes through your API.
POST	`/rooms/:id/images/confirm`	required, owner	Verifies the uploaded object exists, writes `room_images` row.
DELETE	`/rooms/:id/images/:imageId`	required, owner	Deletes DB row + blob.
PATCH	`/rooms/:id/images/:imageId/primary`	required, owner	Unsets existing primary first, in a transaction (schema has a partial unique index on primary).
PATCH	`/rooms/:id/images/reorder`	required, owner	Body: ordered image ID array → updates `sort_order`.

Interests

Method	Path	Auth	Rate limit	Notes
POST	`/rooms/:id/interest`	required	10 / user / 24h (Redis + Postgres trigger backstop)	Reject if `room.owner_id == requester.id`.
GET	`/interests/sent`	required	—	Current user's sent interests.
GET	`/interests/received`	required	—	Owner's lead inbox.
PATCH	`/interests/:id/status`	required, room owner	—	`contacted` / `closed`.

Admin (minimum viable)

Method	Path	Auth	Notes
GET	`/admin/rooms/pending`	admin	Oldest-first review queue.
PATCH	`/admin/rooms/:id/status`	admin	Approve / reject / flag. Reason required on reject/flag.

PHASE 2 — build only after Phase 1 shows real usage

Compare

Design choice: **Redis, not a Postgres table.** Compare lists are short-lived and disposable — persisting them relationally is unnecessary weight. Matches the Redis role already in your architecture.

Method	Path	Auth	Notes
POST	`/compare/:roomId`	required	Adds room to the user's compare set (Redis, key `compare:{userId}`). Cap at 4 rooms — reject the 5th with a clear error, don't silently evict.
DELETE	`/compare/:roomId`	required	Removes one room from the set.
GET	`/compare`	required	Returns full room details (price, photos, distance if `lat`/`lng` passed as query params) for every room in the set.
DELETE	`/compare`	required	Clears the whole list.

Fuller Moderation (extends Phase 1's minimal admin)

Requires new table: **`reports`** (`id`, `room_id`, `reporter_id`, `reason`, `status`, `created_at`) — not in current schema.sql, add before building this.

Method	Path	Auth	Rate limit	Notes
POST	`/rooms/:id/report`	required	10 reports / user / day	Reasons: spam, fraud, misleading, already rented, other. Auto-flag the room (`status = flagged`) after N distinct reports (pick N — e.g. 3 — based on your traffic, don't guess in production).
GET	`/admin/rooms`	admin	—	Full list with filters: `status`, `city`, `reported=true`. Supersedes the Phase 1 pending-only view.
GET	`/admin/reports`	admin	—	Queue of open reports, oldest first.
PATCH	`/admin/reports/:id/resolve`	admin	—	Resolves a report; body includes action taken (dismissed / room flagged / room removed).

OTP & Contact Rate Limiting

Already built and shipped in Phase 1 (moved up deliberately — this was a launch blocker, not a nice-to-have). No new routes here; noting it so this document doesn't imply it's missing.

PHASE 3 — build only if Phase 1 shows retention

Availability Toggle

This resolves the tension flagged earlier: a full `PATCH /rooms/:id` edit resets moderation status, which is correct for content changes but too heavy for a same-day "still available?" flip. This route changes **only** availability, no re-review needed.

Method	Path	Auth	Notes
PATCH	`/rooms/:id/availability`	required, owner	Body: `{ status: "active" | "rented" }` only. Does NOT reset `pending_review`.
POST	`/rooms/:id/renew`	required, owner	Resets the auto-expiry timer (see below) without triggering re-review.

Auto-expiry (system job, not a user-facing route): listings auto-transition to `inactive` after 30 days without a renew. Needs a scheduled job (cron / Azure Function timer trigger), not an API route — noting it here so it isn't lost.

Reviews

Requires new table: **`reviews`** (`id`, `room_id`, `reviewer_id`, `rating`, `comment`, `is_verified`, `created_at`) — not in current schema.sql.

`is_verified` should be set true only if the reviewer has a `contacted`/`closed` row in `interests` for that room — otherwise you get fake reviews from people who never actually engaged, which defeats the point of the feature.

Method	Path	Auth	Rate limit	Notes
POST	`/rooms/:id/reviews`	required	1 review / user / room (unique constraint, same pattern as `interests`)	Rejects if the user has no `interests` record for this room, unless you decide to allow unverified reviews (I'd advise against it).
GET	`/rooms/:id/reviews`	optional	—	Paginated, verified reviews surfaced first.
PATCH	`/reviews/:id`	required, review author	—	Edit own review.
DELETE	`/reviews/:id`	required, author or admin	—	Admin deletion needed for abuse cases.

Saved Searches & Push Notifications

Requires new table: `saved_searches` (`id`, `user_id`, `city`, `room_type`, `min_rent`, `max_rent`, `lat`, `lng`, `radius_km`, `created_at`).

Method	Path	Auth	Notes
POST	`/saved-searches`	required	Creates an alert criteria set.
GET	`/saved-searches`	required	Lists user's own saved searches.
DELETE	`/saved-searches/:id`	required, owner	Removes it.

Matching new listings against saved searches and firing push notifications is a **background job**, not an API route — triggered on `POST /rooms` approval, not on every request. Don't build this as a synchronous check inside the room-creation path; it'll slow down every listing creation for a feature most users won't touch immediately.

Full feature-to-requirement checklist (nothing dropped)

Original requirement	Where it's handled
Login via phone or email	Phase 1 — Auth
Upload multiple room photos	Phase 1 — Room Images
Contact / show interest	Phase 1 — Interests
Compare multiple rooms (price + photos)	Phase 2 — Compare
Live location + nearby distance	Phase 1 — `GET /rooms/nearby`
Fraud/spam prevention (flagged as a risk, not originally requested)	Phase 1 — moderation default, Phase 2 — reports
Rate limiting on OTP + contact (flagged as a risk)	Phase 1 — both routes
Stale listings (flagged as an open question)	Phase 3 — auto-expiry + renew